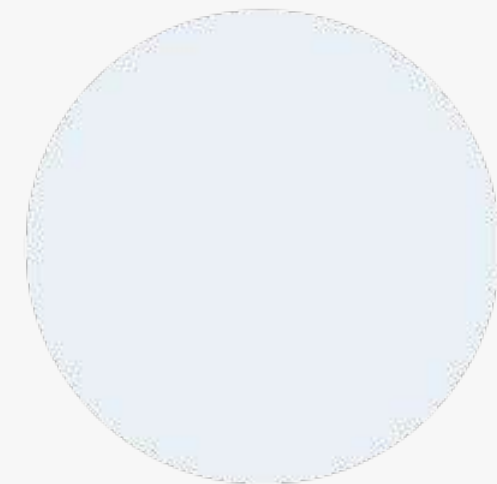


ASTON.



Принципы ООП



astondevs.ru

Наследование



Одним из основополагающих принципов ООП является наследование, который позволяет создать класс (суперкласс), определяющий какие-то общие черты набора классов, а затем этот общий класс может наследоваться другими, более специализированными классами (подклассами), каждый из которых будет добавлять свои особые характеристики. Подкласс наследует члены, определённые в суперклассе, добавляя к ним собственные.

NB: Важно! Подкласс будет наследовать члены, определённые в суперклассе, в соответствии с модификаторами доступа этих членов. Если у суперкласса будет `private` поле, то подкласс не унаследует это поле.

Для реализации наследования используется ключевое слово `extends` в следующей форме:

```
class имя_подкласса extends имя_суперкласса
```

Наследование



Представим, что в нашем приложении нам придется работать с различными животными (считаем что они домашние), у всех этих животных должна быть кличка (name), и все они должны уметь прыгать. Если представить, что нам нужно штук 10 разных классов животных, то поле name и метод jump() надо будет в каждом из них прописывать, тем самым дублируя один и тот же код (а если одинаковых полей и методов больше?). Вместо дублирования кода, мы можем создать суперкласс Animal, в котором и описать эти общие для всех подклассов черты.

После чего создавать подклассы и наследоваться от класса Animal. В приведенном ниже примере представлена только структура Animal и Cat классов, можно мысленно представить что помимо Cat у нас еще есть Dog, Hamster, и т.д.

NB: Заметка. В приведенном примере специально оставлены конструктор по-умолчанию для класса Animal и методы animalInfo() и catInfo(). Эти части кода в дальнейшем немного оптимизируются, как только мы разберемся в соответствующих темах.

```
public class Animal {  
    String name;  
    public Animal() {  
    }  
    public Animal(String name) {  
        this.name = name;  
    }  
    public void animalInfo() {  
        System.out.println("Животное: " +  
name);  
    }  
    public void jump() {  
        System.out.println("Животное  
подпрыгнуло");  
    }  
}
```

Наследование



Подкласс Cat включает в себя все члены своего суперкласса Animal. Именно поэтому объект cat имеет доступ к методу animalInfo(), и в методе catInfo() возможна непосредственная ссылка на переменную color, как если бы она была частью класса Cat.

В свою очередь у кота появилось такое свойство как цвет (color) и метод catInfo(), которых нет в суперклассе Animal.

Несмотря на то что класс Animal является суперклассом для класса Cat, он в то же время остаётся полностью независимым и самостоятельным классом. То, что один класс является суперклассом для другого, совсем не исключает возможность его самостоятельного использования.

NB: Важно! Для каждого создаваемого класса можно указать только один суперкласс — в Java не поддерживается множественное наследование. Если суперкласс не указан явно, то класс наследуется от класса java.lang.Object (в приведенном выше примере, класс Animal является подклассом суперкласса Object)

Более того, один подкласс может быть суперклассом другого подкласса.

NB: Важно! Абсолютно все классы в Java являются прямыми или косвенными наследниками класса Object (из пакета java.lang). Cat является подклассом Animal, а Animal подкласс Object, следовательно Cat тоже подкласс Object.

```
public class Cat extends Animal {
    String color;
    public Cat(String name, String color) {
        this.name = name;
        this.color = color;
    }
    public void catInfo() {
        System.out.println("Кот имя: " + name
+ " цвет: " + color);
    }
}

public class AnimalsApp {
    public static void main(String[] args) {
        Animal animal = new Animal("Дружок");
        Cat cat = new Cat("Барсик", "Белый");
        animal.animalInfo();
        cat.animalInfo();
        cat.catInfo();
    }
}

// Результат:
// Животное: Дружок
// Животное: Барсик
// Кот имя: Барсик цвет: Белый
```

Ключевое слово super



У ключевого слова `super` имеются две общие формы. Первая служит для вызова конструктора суперкласса, вторая — для обращения к члену суперкласса, скрываемому членом подкласса.

Из подкласса можно вызывать конструктор, определенный в его суперклассе, используя следующую форму ключевого слова `super`:

```
super(список_аргументов)
```

Здесь `список_аргументов` определяет аргументы, требующиеся конструктору суперкласса.

NB: Важно! Вызов метода `super()` всегда должен быть первым оператором, выполняемым в конструкторе подкласса. Если в конструкторе подкласса явно не использовать `super()`, то автоматически первой строкой будет вызываться конструктор по умолчанию из суперкласса.

Такая конструкция позволяет заполнять даже поля суперкласса с модификатором доступа `private`. Например:

```
public class Animal {
    private int a;
    protected int z;
    public Animal(int a) {
        this.a = a;
    }
}

public class Cat extends Animal {
    private int b;
    protected int z;
    public Cat(int a, int b) {
        super(a); // первым делом вызываем
конструктор Animal
        this.b = b;
    }
    public void test() {
        z = 10; // Обращение к полю z класса Cat
        super.z = 20; // Обращение к полю z класса
Animal
    }
}

public class BombayCat extends Cat {
    private int c;
    public BombayCat(int a, int b, int c) {
        super(a, b); // первым делом вызываем
конструктор Cat this.c = c;
    }
}
```


Ключевое слово `super`



Вторая форма применения ключевого слова `super` действует подобно ключевому слову `this`, за исключением того, что ссылка всегда делается на суперкласс. Вторая форма наиболее пригодна в тех случаях, когда имена членов подкласса скрывают члены суперкласса с такими же именами, в примере выше поле `z` класса `Cat` скрывает поле `z` суперкласса, поэтому для доступа к полю суперкласса используется запись `super.z`. То же справедливо и для методов.

NB: Важно! Если вы только начинаете программировать, то **КРАЙНЕ НЕ РЕКОМЕНДУЕТСЯ** объявлять поля с одинаковыми именами в суперклассе и его подклассах, потому как очень легко будет запутаться с каким из полей вы работаете. Такое объявление переменных имеет только если вы без этого никак не можете обойтись, и абсолютно четко понимаете что делаете.

Порядок вызова конструкторов



При вызове конструктора `BombeyCat` будут по цепочке вызваны конструкторы родительских классов, начиная с самого первого класса.

```
BombeyCat bombeyCat = new BombeyCat ();  
// Animal() => Cat() => BombeyCat()
```

Конструкторы вызываются в порядке наследования, поскольку суперклассу ничего неизвестно о своих подклассах, и поэтому любая инициализация должна быть выполнена в нём совершенно независимо от любой инициализации, выполняемой подклассом. Следовательно, она должна выполняться в первую очередь.

Ключевое слово `final` в сочетании с наследованием



Существует несколько способов использования ключевого слова `final`:
Первый способ: создание именованной константы.

```
final int MONTHS_COUNT = 12;  
// final в объявлении поля или переменной
```

Второй способ: предотвращение переопределения методов.

```
public final void run() { }  
// final в объявлении метода
```

Третий способ: запрет наследования от текущего класса.

```
public final class A { // final в объявлении класса  
}
```

```
// public class B extends A { // Ошибка, класс A не может  
// иметь подклассы  
// }
```

Внутренние классы имеют право наследовать другие классы, реализовывать интерфейсы и выступать в роли объектов наследования.

Наследование от внешнего класса

```
public class SubAddress extends Address {  
// code  
}  
}
```


Наследование от внутреннего класса



Если внутренний класс наследуется обычным образом другим классом (после `extends` указывается `ИмяВнешнегоКласса.ИмяВнутреннегоКласса`), то он теряет доступ к полям своего внешнего класса, в котором был объявлен.

```
public class FreeAddress extends Student.Address {  
    public FreeAddress() {  
        new Student().super();  
    }  
    public FreeAddress(Student student) {  
        student.super();  
    }  
}
```

В данном случае конструктор класса `FreeAddress` должен объявлять объект класса `Student` или получать его в виде параметра, что позволит получить доступ к ссылке на внутренний класс `Address`, наследуемый классом `FreeAddress`.

Внутренние классы позволяют решить проблему множественного наследования сущности, когда требуется наследовать свойства нескольких классов.

При объявлении внутреннего класса могут использоваться модификаторы `final`, `abstract`, `private`, `protected`, `public`.

Поля внешнего класса видны внутреннему классу так, будто они его собственные, модификаторы видимости игнорируются. Применить ссылку `this` также не получится, так как `this` внутри класса `Inner` указывает на его собственный объект, и ни в коем случае не на его владельца.

Интерфейсы



Назначение интерфейса — описание или спецификация функциональности, которую должен реализовывать каждый класс, его имплементирующий. Класс, реализующий интерфейс, предоставляет к использованию объявленный интерфейс в виде набора public-методов в полном объеме.

Интерфейсы подобны абстрактным классам, хотя и не являются классами. В языке Java существует три вида интерфейсов:

- интерфейсы, определяющие функциональность для классов посредством описания методов, но не их реализации;
- функциональные интерфейсы, специфицирующие в одном абстрактном методе свое применение;
- интерфейсы, реализация которых автоматически придает классу определенные свойства.

К последним относятся, например, интерфейсы Cloneable, Serializable, Externalizable, отвечающие за клонирование и сохранение объекта (сериализацию) в информационном потоке соответственно. Общее определение интерфейса имеет вид:

```
[public] interface Name [extends
NameOtherInterface, ..., NameN] {
    // constants, methods
}
```

Все объявленные в интерфейсе абстрактные методы автоматически трактуются как public abstract, а все поля — как public static final, даже если они так не объявлены. Интерфейсы могут объявлять статические методы. В интерфейсах могут объявляться методы с реализацией с ключевым словом default. Эти методы могут быть public или private.

Класс может реализовывать любое число интерфейсов, указываемых через запятую после ключевого слова implements, дополняющего определение класса. После этого класс обязан реализовать все абстрактные методы, полученные им от интерфейсов, или объявить себя абстрактным классом.

Интерфейсы



Если необходимо определить набор функциональности для какого-либо рода деятельности, например, для управления счетом в банке, то следует использовать интерфейс вида:

```
public interface AccountAction{  
    boolean openAccount();  
    boolean closeAccount();  
    void blocking();  
    default void unBlocking() {}  
    double depositInCash(int accountId, int amount);  
    boolean withdraw(int accountId, int amount);  
    boolean convert(double amount);  
    boolean transfer(int accountId, double amount);  
}
```

В интерфейсе обозначены, но не реализованы, действия, которые может производить клиент со своим счетом. Реализация не представлена из-за возможности различного способа выполнения действия в конкретной ситуации.

А именно: счет может блокироваться автоматически, по требованию клиента или администратором банковской системы. В каждом из трех указанных случаев реализация метода `blocking()` будет уникальной и никакого базового решения предложить невозможно. С другой стороны, наличие в интерфейсе метода заставляет класс, его имплементирующий, предоставить реализацию методу. Программист получает повод задуматься о способе реализации функциональности, так как наличие метода в интерфейсе говорит о необходимости той или иной функциональности всем классам, реализующим данный интерфейс.

Метод **`unblocking()`** предоставляет дефолтную реализацию метода, которая может быть при необходимости переопределена в подклассе.

Интерфейсы



Если по каким-либо причинам для данного класса не предусмотрена реализация метода или его реализация нежелательна, рекомендуется генерация непроверяемого исключения в теле метода, а именно:

```
public boolean blocking() {  
    throw new UnsupportedOperationException();  
}
```

Менее хорошим примером будет реализация в виде:

```
public boolean blocking() {  
    return false;  
}
```

В интерфейсе не могут быть объявлены поля без инициализации, интерфейс в качестве полей содержит только final static константы. Множественное наследование между интерфейсами допустимо. Классы, в свою очередь, интерфейсы только реализуют. Класс может наследовать один суперкласс и реализовывать произвольное число интерфейсов.

Интерфейсы обычно объявляются как public, потому что описание функциональности, предоставляемое ими, может быть использовано в других пакетах проекта. Интерфейсы с областью видимости в рамках пакета, атрибут доступа по умолчанию, могут использоваться только в этом пакете и нигде более.

Реализация интерфейсов классом может иметь вид:

```
[public] class NameClass implements Name1,  
Name2, ..., NameN {  
    /* */  
}
```

Здесь Name1, Name2,..., NameN — перечень используемых интерфейсов. Класс, который реализует интерфейс, должен предоставить полную реализацию всех абстрактных методов, объявленных в интерфейсе. Кроме того, данный класс может объявлять свои собственные методы.

При неполной реализации интерфейса класс должен быть объявлен как абстрактный, что говорит о необходимости реализации абстрактных методов уже в его подклассе.

Полиморфизм



Полиморфизм – это способность программы идентично использовать объекты с одинаковым интерфейсом без информации о конкретном типе этого объекта.

Данный принцип позволяет программистам использовать одни и те же термины для описания различного поведения, зависящего от контекста.

Одной из форм полиморфизма в Java является переопределение метода, когда различные формы поведения определяются объектом из которого данный метод был вызван.

Другой формой полиморфизма является перегрузка метода, когда его поведение определяется набором передаваемых в метод аргументов.

Переопределение методов



В случае с переопределением метода, дочерний класс, используя концепцию полиморфизма, может изменить (переопределить) поведение метода родительского класса. Это позволяет программисту по-разному использовать один и тот же метод, определяя поведение из контекста вызова (вызывается метод из класса предка или класса наследника). Говорят, что метод из подкласса переопределяет метод из суперкласса, если у супер- и подкласса совпадают сигнатуры методов. Когда переопределённый метод вызывается из своего подкласса, он всегда ссылается на свой вариант, определённый в подклассе. А вариант метода, определённого в суперклассе, будет скрыт.

Пусть любое животное в нашем приложении должно уметь подавать голос, по-умолчанию при вызове метода `voice()` мы будем видеть стандартное сообщение, что животное издало какой-то звук. Для тех же классов, которые издают вполне конкретные звуки, мы можем переопределить метод `voice()`, и например, конкретизировать что `Cat` именно мяукает, а не что-то еще делает.

```
public class Animal {
    void voice() {
        System.out.println("Животное издало какой-то звук");
    }
}

public class Dog extends Animal {
}

public class Cat extends Animal {
    @Override
    void voice() {
        System.out.println("Кот мяукнул");
    }
}

public class AnimalsApp {
    public static void main(String[] args) {
        Animal animal = new Animal();
        Cat cat = new Cat();
        Dog dog = new Dog();
        animal.voice();
        cat.voice();
        dog.voice();
    }
}

// Результат:
// Животное издало звук
// Кот мяукнул
// Животное издало звук
```

Переопределение методов



Когда метод `voice()` вызывает объект типа `Cat`, выбирается вариант этого метода, определённый в классе `Cat`. У объекта же класса `Dog` своей реализации метода `voice()` не было, поэтому `dog` выполнил вариант метода `voice()`, который описан в суперклассе `Animal`.

NB: Важно! Над методами подклассов, переопределяющими методы суперклассов, можно ставить аннотацию `@Override`, но она не является обязательной. Она всего лишь проверит, действительно ли в родительском классе есть такой метод, который вы собрались переопределять.

Если при переопределении метода необходим функционал из этого метода суперкласса, то можно использовать конструкцию `super.method()`. Пример и результат использования:

NB: Важно! Переопределение методов выполняется только в том случае, если имя, список аргументов и возвращаемый тип обоих методов одинаковы. В противном случае оба метода считаются перегруженными.

```
public class Cat extends Animal {
    @Override
    public void voice() {
        super.voice(); // вызываем метод
        voice() суперкласса
        System.out.println("Кот мяукнул");
    }
}

public class CatsApp {
    public static void main(String[] args) {
        Cat cat = new Cat();
        cat.voice();
    }
}

// Результат:
// Животное издало звук
// Кот мяукнул
```

Переопределение методов



Переопределенные методы позволяют поддерживать в Java полиморфизм во время выполнения, он позволяет определить в общем классе методы, которые станут общими для всех производных от него классов, а в подклассах — конкретные реализации некоторых или всех этих методов. При работе с суперклассами и подклассами существует возможность создать ссылку на суперкласс и записать в нее объект подкласса:

```
public class DemoApp {  
    public static void main(String[] args) {  
        Animal animal = new Cat();  
        animal.voice();  
        if (animal instanceof Cat) {  
            ((Cat) animal).methodFromCatClass();  
            System.out.println("В animal действительно лежит  
кот");  
        }  
    }  
}
```

Несмотря на то, что объект типа Cat лежит переменной типа Animal, реализацию метода voice() он будет брать именно ту, которая ближе к нему, то есть описанная в классе Cat. При обращении к объекту типа Cat через ссылку на Animal мы будем видеть только те методы и поля, которые предоставляет нам класс Animal.

Переопределение методов



Если же у нас в классе `Cat` есть некий метод `methodFromCatClass()` и мы захотим его все же выполнить через переменную `animal`, необходимо явно указать класс с которым мы работаем: `((Cat)animal)`. После чего мы сможем пользоваться методами и полями из класса `Cat`.

Если в `animal` будет лежать ссылка не на объект типа `Cat` и вы используется запись вида `((Cat)animal)`, это операция приведет к ошибке в работе программы (исключению `ClassCastException`). Чтобы избежать такой ошибки можно воспользоваться оператором `instanceof`, который проверяет принадлежность объекта к какому-либо классу.

Ограничения при переопределении



У переопределения методов класса в Java есть ряд ограничений:

- Название метода должно быть таким же, как у метода родителя (то есть сигнатура метода должна быть одинаковой).
- Аргументы должны оставаться такими же, как у метода родителя.
- Окончательные методы (`final`) нельзя переопределять. Это один из способов запрета переопределения — объявить метод с помощью ключевого слова `final`.
- Статические методы (`static`) нельзя переопределять. Если вы определите в классе-наследнике такую же сигнатуру метода, как в родительском классе, то выполните сокрытие.
- Приватные методы (`private`) нельзя переопределять, так как они связываются на этапе компиляции, а не выполнения.
- Нельзя сужать модификатор доступа — например, с `public` до `private`. Расширение уровня доступа возможно.
- Нельзя менять тип возвращаемого значения, однако можно сузить возвращаемое значение, если они совместимы.

Перегрузка методов



В случае с перегрузкой, метод может проявлять различное поведение в зависимости от того, какие аргументы он принимает. В данном случае контекст вызова определяется набором параметров метода.

Перегрузка позволяет нашей программе быть более гибкой и учитывать различные варианты работы.

```
public class Jarvis {  
    public void sayHi(String firstGuest) {  
        System.out.println("Добрый вечер, " + firstGuest + ",  
как ваши дела?");  
    }  
    public void sayHi(String firstGuest, String secondGuest)  
{  
        System.out.println("Добрый вечер, " + firstGuest + ",  
" + secondGuest + ", как ваши дела?");  
    }  
    public static void main(String[] args) {  
        Jarvis jarvis = new Jarvis();  
        jarvis.sayHi("Тони Старк");  
        jarvis.sayHi("Тони Старк", "Капитан Америка");  
    }  
}
```

Добрый вечер, Тони Старк, как ваши дела?

Добрый вечер, Тони Старк, Капитан Америка, как ваши дела?

Если у методов одинаковые имена, как Java узнает, какой именно из них вызывается?

Ответ прост: перегружаемые методы должны отличаться по типу и/или количеству входных параметров. Даже разного порядка аргументов достаточно для того, чтобы методы считались разными (хотя это не рекомендуется).

Логично спросить, почему при перегрузке используются только имена классов и списки аргументов? Почему не идентифицировать методы по их возвращаемым значениям?

```
// ДАННЫЙ КОД ВЫЗОВЕТ ОШИБКУ КОМПИЛЯЦИИ!  
class MyClass {  
    public int foo() {  
        return 0;  
    }  
    public double foo() {  
        return 0;  
    }  
}
```

Перегрузка методов



Идентифицировать их нельзя, потому что Java в этом случае не может определить, какая версия метода должна выполняться.

При вызове перегружаемого метода для определения нужного варианта в Java используется тип и/или количество аргументов метода.

Следовательно, перегружаемые методы должны отличаться по типу и/или количеству их параметров. Возвращаемые типы перегружаемых методов могут отличаться, но самого возвращаемого метода недостаточно, чтобы отличить два разных варианта метода.

Когда в исполняющей среде Java встречается вызов перегружаемого метода, в ней просто выполняется тот вариант, параметры которого соответствуют аргументам, указанным в вызове.

Перегрузка методов позволяет поддерживать принцип «один интерфейс, несколько методов»



В языках программирования без перегрузки методов, каждому методу должно быть присвоено однозначное имя. Но зачастую требуется реализовать, по существу, один и тот же метод для разных типов данных.

В таком случае, в языках программирования без перегрузки реализуют несколько методов, которые немного отличаются названиями.

Перегрузка методов ценна тем, что позволяет обращаться к похожим методам по общему имени. Следовательно, имя представляет общее действие, которое должно выполняться.

Выбор подходящего варианта метода для конкретной ситуации входит в обязанности компилятора.

Ничто не запрещает вам реализовать несколько перегруженных методов, каждый из которых будет работать совершенно по-разному. Но на практике крайне рекомендуется, чтобы перегруженные методы реализовывали одну и ту же общую операцию.

Перегрузка методов позволяет поддерживать принцип «один интерфейс, несколько методов»



Большим преимуществом и целью использования полиморфизма является уменьшение связанности клиентского класса с реализацией. Вместо того чтобы хардкодить, клиентский класс получает реализацию зависимости для выполнения необходимого действия.

Таким образом, клиентский класс знает минимум для выполнения своих действий, что является примером слабого связывания.

Существуют определенные требования к объявлению методов в базовых классах, чтобы не нарушался принцип полиморфизма. Эти методы не должны быть `static`, `private` и `final`.

- **Private** делает метод доступным только в классе, и вы не сможете его переопределить в наследнике.
- **Static** делает метод достоянием класса, а не объекта, поэтому всегда будет вызываться метод суперкласса.
- **Final** же сделает метод неизменяемым и скрытым от наследников.

Полиморфизм



Что нужно помнить о **полиморфизме**:

- Созданный экземпляр определяет, какой метод будет вызван при использовании полиморфизма.
- Аннотация @Override обязывает программиста использовать переопределенный метод; в противном случае возникнет ошибка компилятора.
- Полиморфизм может использоваться с обычными классами, абстрактными классами и интерфейсами.
- Большинство шаблонов проектирования зависят от той или иной формы полиморфизма.
- Единственный способ вызвать нужный ваш метод в полиморфном подклассе — это использовать приведение типов.

Абстракция



Абстракция означает использование простых вещей для описания чего-то сложного. Например, мы все знаем как пользоваться телевизором, но в тоже время нам не нужно обладать знаниями о том, как он работает чтобы смотреть его.

```
abstract void voice();
```

В Java под абстракцией подразумеваются такие вещи, как объекты, классы и переменные, которые в свою очередь лежат в основе более сложного кода. Использование данного принципа позволяет избежать сложности при разработке ПО.

Иногда суперкласс требуется определить таким образом, чтобы объявить в нём структуру заданной абстракции, не предоставляя полную реализацию каждого метода. Например, определение метода `voice()` в классе `Animal` служит лишь в качестве шаблона, поскольку все животные издают разные звуки, а значит нет возможности прописать хоть какую-то реализацию этого метода в классе `Animal`. Для этой цели служит абстрактный метод (с модификатором `abstract`). Иногда они называются методами под ответственностью подкласса, поскольку в суперклассе для них никакой реализации не предусмотрено, и они обязательно должны быть переопределены в подклассе.

Абстракция



При указании ключевого слова `abstract` в объявлении метода, тело метода будет отсутствовать. Класс, содержащий хоть один абстрактный метод, должен быть объявлен как абстрактный (в объявлении класса также добавляется ключевое слово `abstract`).

Нельзя создавать объекты абстрактного класса, поскольку он определен не полностью. Кроме того, нельзя объявлять абстрактные конструкторы или абстрактные статические методы.

Любой подкласс, производный от абстрактного класса, обязан реализовать все абстрактные методы из своего суперкласса (при условии что подкласс сам не является абстрактным). При этом абстрактный класс вполне может содержать конкретные реализации методов. Пример:

```
public abstract class Animal {  
    public abstract void voice();  
    public void jump() {  
        System.out.println("Животное  
подпрыгнуло");  
    }  
}  
  
public class Cat extends Animal {  
    @Override  
    public void voice() {  
        System.out.println("Кот мяукнул");  
    }  
}
```

Абстракция



NB: Важно! Что нужно помнить об абстрактных классах:

```
Animal a = new Cat();
```

- Нельзя создать объект абстрактного класса;
- В абстрактном классе могут быть конкретные реализации методов;
- Если в классе есть хоть один абстрактный метод, он должен быть объявлен абстрактным;
- Несмотря на то, что абстрактные классы не позволяют получать экземпляры объектов, их всё же можно применять для создания ссылок на объекты подклассов.

Инкапсуляция



Инкапсуляция связывает данные с манипулирующим ими кодом и позволяет управлять доступом к членам класса из отдельных частей программы, предоставляя доступ только с помощью определенного ряда методов, что позволяет предотвратить злоупотребление этими данными.

То есть класс должен представлять собой «черный ящик», которым можно пользоваться, но его внутренний механизм защищен от повреждений.

Способ доступа к члену класса определяется модификатором доступа, присутствующим в его объявлении. Некоторые аспекты управления доступом связаны, главным образом, с наследованием и пакетами, и будут рассмотрены позднее. В Java определяются следующие модификаторы доступа: `public`, `private` и `protected`, а также уровень доступа, предоставляемый по умолчанию.

Инкапсуляция



Любой `public` член класса доступен из любой части программы. Компонент, объявленный как `private`, доступен только внутри класса, в котором объявлен. Если в объявлении члена класса отсутствует явно указанный модификатор доступа (`default`), то он доступен для подклассов и других классов из данного пакета. Если же требуется, чтобы элемент был доступен за пределами его текущего пакета, но только классам, непосредственно производным от данного класса, то такой элемент должен быть объявлен как `protected`.

Модификатор доступа предшествует остальной спецификации типа члена.

```
public int num;  
protected char symb;  
boolean active;  
private void calculate(float x1, float x2) {  
    // ...  
}
```

Как правило, доступ к данным объекта должен осуществляться только через методы, определённые в классе этого объекта. Поле экземпляра вполне может быть открытым, но на то должны иметься веские основания. Для доступа к данным обычно используются методы: геттеры и сеттеры.

Геттер позволяет узнать содержимое поля, его тип совпадает с типом поля для которого он создан, а имя, как правило, начинается со слова `get`, к которому добавляется имя поля.

Сеттер используется для изменения значения поля, имеет тип `void` и именуется по аналогии с геттером, только `get` заменяется на `set`. Сеттер позволяет добавлять ограничения на изменение полей — в примере ниже с помощью сеттера не получится указать коту отрицательный или нулевой возраст.

Инкапсуляция



```
public class Cat {  
    private String name;  
    private int age;  
    public void setAge(int age) {  
        if (age >= 0) {  
            this.age = age;  
        } else {  
            System.out.println("Введен некорректный  
возраст");  
        }  
    }  
    public int getAge() {  
        return age;  
    }  
    public void setName(String name) {  
        this.name = name;  
    }  
    public String getName() {  
        return name;  
    }  
}
```

NB: Заметка. Если для поля создан только геттер, то вне класса это поле будет доступно только для чтения. Если ни создан ни геттер, ни сеттер, то работа с полем снаружи класса может осуществляться только косвенно, через другие методы этого класса. (Пусть в классе Cat есть поле вес, оно private и для него нет геттеров и сеттеров. Тогда мы не можем через сеттер изменить вес кота напрямую, но если мы его покормим, то кот может сам набрать вес. Мы не можем запросить вес через геттер и получить конкретное значение, но у кота может быть метод info(), который выведет в консоль нам величину, записанную в поле weight.).

Инкапсуляция



Особенности всех уровней доступа в языке Java сведены в таблицу:

	private	Модификатор отсутствует	protected	public
Один и тот же класс	+	+	+	+
Подкласс, производный от класса из того же самого пакета	-	+	+	+
Класс из того же самого пакета, не являющийся подклассом	-	+	+	+
Подкласс, производный от класса другого пакета	-	-	+	+
Класс из другого пакета, не являющийся подклассом, производный от класса данного пакета	-	-	-	+

Ключевое слово **static**



Иногда возникает необходимость создать поле класса, общее для всех объектов этого класса, или метод, который можно было бы использовать без необходимости создания объектов класса, в котором он прописан.

Обращение к такому полю или методу должно осуществляться через имя класса. Для этого при объявлении поля или метода указывается ключевое слово **static**. Когда член класса объявлен как **static** (статический), он доступен до создания любых объектов его класса и без ссылки на конкретный объект.

Наиболее распространённым примером статического члена служит метод `main()`. При создании объектов класса копии статических полей не создаются и все объекты этого класса используют одно и то же статическое поле.

На методы, объявленные как **static**, накладываются следующие ограничения:

- Они могут непосредственно вызывать только другие статические методы.
- Им непосредственно доступны только статические переменные.
- Они никоим образом не могут использовать ссылки типа `this` или `super`.

ПРАКТИЧЕСКОЕ ЗАДАНИЕ



Задание 1:

Создать классы Собака и Кот с наследованием от класса Животное.

Все животные могут бежать и плавать. В качестве параметра каждому методу передается длина препятствия. Результатом выполнения действия будет печать в консоль. (Например, `dogBobik.run(150);` → 'Бобик пробежал 150 м.');

У каждого животного есть ограничения на действия (бег: кот 200 м., собака 500 м.; плавание: кот не умеет плавать, собака 10 м.).

Добавить подсчет созданных котиков, собак и животных.

Расширить задачу про котиков и тарелки с едой, выполнив следующие пункты:

- Сделать так, чтобы в тарелке с едой не могло получиться отрицательного количества еды (например, в миске 10 еды, а кот пытается покушать 15-20).
- Каждому коту нужно добавить поле сытость (когда создаем котиков, они голодны). Если коту удалось покушать (хватило еды), сытость = true.
- Считаем, что если коту мало еды в тарелке, то он её просто не трогает, то есть не может быть наполовину сыт (это сделано для упрощения логики программы).
- Создать массив котиков и тарелку с едой, попросить всех котиков покушать из этой тарелки и потом вывести информацию о сытости котиков в консоль.
- Добавить в тарелку метод, с помощью которого можно было бы добавлять еду в тарелку.

ПРАКТИЧЕСКОЕ ЗАДАНИЕ



Задание 2:

Применяя интерфейсы написать программу расчета периметра и площади геометрических фигур: круг, прямоугольник, треугольник.

Задать для каждой фигуры цвет заливки и цвет границы.

Результат полученных характеристик [Периметр, площадь, цвет фона, цвет границ] по каждой фигуре вывести в консоль.

Попробуйте реализовать базовые методы, такие как расчет периметра фигур, в качестве дефолтных методов в интерфейсе.